

Docker Containerization

Description

1. Research and study the basics of Docker Containerization platform.
2. Install Docker on your machine.
3. Build a container from the available Docker images.
4. Study basic Docker commands.

1. Introduction to Docker

Docker is a **containerization platform** used to package and run applications in isolated environments called **containers**.

A Docker container contains the application and the required dependencies needed to run it.

Basic Docker Concepts

- **Docker Image** – A read-only template used to create containers.
- **Docker Container** – A running instance of a Docker image.
- **Dockerfile** – A file containing instructions for creating a custom Docker image.
- **Docker Hub** – A public registry from which Docker images can be downloaded.
- **Docker Volume** – Persistent storage used to keep data even after a container is removed.
- **Port Mapping** – Connects a port on the host machine to a port inside the container.

Basic Docker Flow

Docker Image
↓
Docker Container
↓
Running Application

2. Docker Installation

The Docker environment was configured on a **RHEL 10.2** EC2 instance.

System Update

```
sudo dnf update -y
```

Install Docker Repository Management Tools

```
sudo dnf install -y dnf-plugins-core
```

Add Docker Repository

```
sudo dnf config-manager --add-repo  
https://download.docker.com/linux/rhel/docker-ce.repo
```

Install Docker

```
sudo dnf install -y docker-ce docker-ce-cli containerd.io docker-  
buildx-plugin docker-compose-plugin
```

Docker Service Issue

Initially, Docker failed to start because required kernel networking modules were not available.

The system was rebooted after confirming that the newer kernel was installed.

After rebooting, the kernel version was verified:

```
uname -r
```

The newer kernel was:

```
6.12.0-211.61.1.el10_2.x86_64
```

The required modules were loaded:

```
sudo modprobe br_netfilter  
sudo modprobe xt_addrtype
```

The modules were verified using:

```
lsmod | grep -E 'br_netfilter|xt_addrtype'
```

Docker was then started successfully:

```
sudo systemctl start docker
```

Docker was enabled to start automatically:

```
sudo systemctl enable docker
```

Docker Version Verification

```
docker --version
```

Output:

```
Docker version 29.8.2, build 7fc2dff
```

Docker information was also checked:

```
sudo docker info
```

This confirmed that the Docker Engine was running correctly.

```
[ec2-user@ip-172-31-2-73 ~]$ docker --version
Docker version 29.8.2, build 7fc2dff
[ec2-user@ip-172-31-2-73 ~]$ sudo docker info
Client: Docker Engine - Community
Version:      29.8.2
Context:      default
Debug Mode:   false
Plugins:
  buildx: Docker Buildx (Docker Inc.)
    Version:  v0.37.1
    Path:      /usr/libexec/docker/cli-plugins/docker-buildx
  compose: Docker Compose (Docker Inc.)
    Version:   v5.5.1
    Path:      /usr/libexec/docker/cli-plugins/docker-compose

Server:
Containers: 0
  Running: 0
  Paused: 0
  Stopped: 0
Images: 0
Server Version: 29.8.2
Storage Driver: overlayfs
  driver-type: io.containerd.snapshotter.v1
Logging Driver: json-file
Cgroup Driver: systemd
Cgroup Version: 2
Plugins:
  Volume: local
  Network: bridge host ipvlan macvlan null overlay
  Log: awslogs fluentd gcplogs gelf journald json-file local splunk syslog
CDI spec directories:
  /etc/cdi
  /var/run/cdi
Swarm: inactive
Runtimes: io.containerd.runc.v2 runc
Default Runtime: runc
Init Binary: docker-init
containerd version: ee2735368117d2eb259779949d5e75cdafec9761
runc version: v1.5.1-0-g8f2685a4
init version: de40ad0
Security Options:
```

3. Hosting Apache Web Server Using a Ready-Made Docker Image

The official Apache HTTP Server image was used to create a container.

Pull Apache Image

```
sudo docker pull httpd
```

The httpd image was downloaded from Docker Hub.

Create Apache Container

```
sudo docker run -d --name myapache -p 8080:80 httpd
```

Here:

- `-d` runs the container in detached mode.
- `--name myapache` gives the container the name `myapache`.
- `-p 8080:80` maps host port `8080` to container port `80`.
- `httpd` is the Apache Docker image.

Verify Container

```
sudo docker ps
```

The Apache container was running and port `8080` was mapped to port `80` inside the container.

Configure Webpage

The Apache container was accessed:

```
sudo docker exec -it myapache bash
```

The Apache web directory was accessed:

```
cd /usr/local/apache2/htdocs
```

The `index.html` file was created with the following content:

```
<!DOCTYPE html>
<html>
<head>
  <title>My Docker Apache Server</title>
</head>
<body>
  <h1>Hello from Docker!</h1>
  <h2>Apache Web Server is running inside a Docker Container</h2>
  <p>This webpage is being served by Apache running in Docker.</p>
  <p>Container Name: myapache</p>
</body>
</html>
```

The container was exited:

exit

The webpage was accessed using:

<http://13.201.129.84:8080>

Apache Webpage Hosted

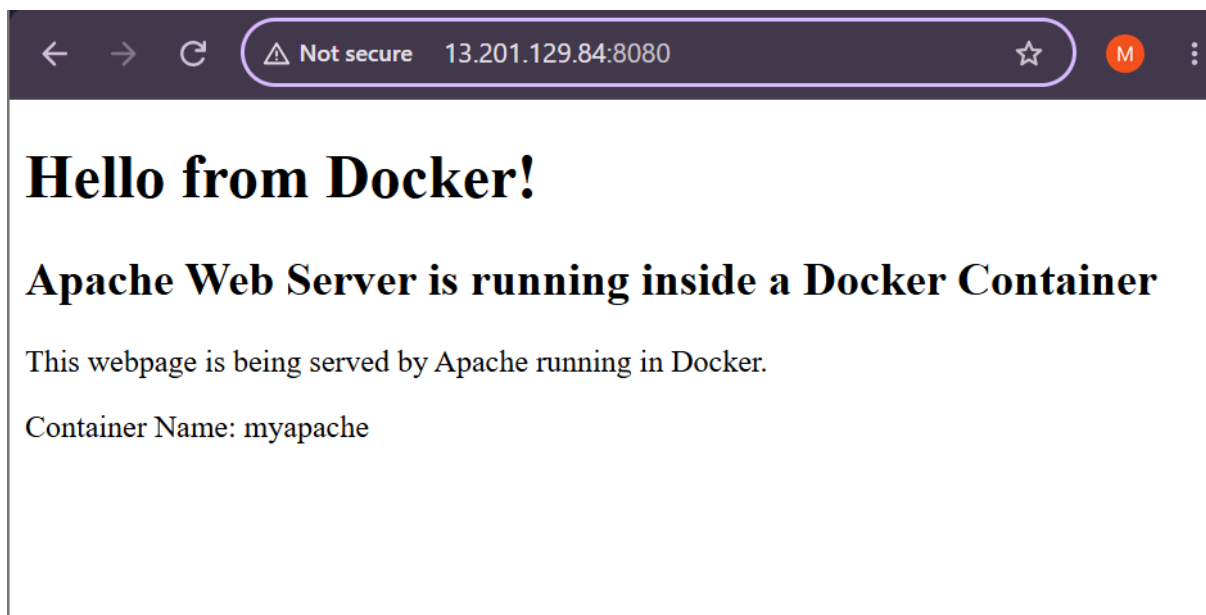
The following webpage was successfully displayed:

Hello from Docker!

Apache Web Server is running inside a Docker Container

This webpage is being served by Apache running in Docker.

Container Name: myapache



4. Basic Docker Commands

The following basic Docker commands were studied and used.

Check Docker Version

```
docker --version
```

Display Docker System Information

```
sudo docker info
```

Download an Image

```
sudo docker pull IMAGE
```

List Docker Images

```
sudo docker images
```

Create and Run a Container

```
sudo docker run -d --name NAME -p HOST_PORT:CONTAINER_PORT IMAGE
```

List Running Containers

```
sudo docker ps
```

List All Containers

```
sudo docker ps -a
```

Start a Container

```
sudo docker start CONTAINER
```

Stop a Container

```
sudo docker stop CONTAINER
```

Restart a Container

```
sudo docker restart CONTAINER
```

View Container Logs

```
sudo docker logs CONTAINER
```

Access a Running Container

```
sudo docker exec -it CONTAINER bash
```

Inspect a Container

```
sudo docker inspect CONTAINER
```

Rename a Container

```
sudo docker rename OLD_NAME NEW_NAME
```

Remove a Container

```
sudo docker rm CONTAINER
```

Remove an Image

```
sudo docker rmi IMAGE
```

5. Creating a Custom Nginx Docker Image

A custom Docker image was created using Nginx as the base image.

Create Working Directory

```
mkdir ~/mynginx-image  
cd ~/mynginx-image
```

Create Custom Webpage

An index.html file was created with the following content:

```
<!DOCTYPE html>  
<html>  
<head>  
    <title>My Custom Nginx Docker</title>  
</head>  
<body>  
    <h1>Welcome to My Nginx Server</h1>  
    <h2>My Own Docker Image</h2>  
    <p>This webpage is running from a custom Nginx Docker image.</p>  
    <p>Created as part of my Docker containerization lab.</p>  
</body>  
</html>
```


Create Dockerfile

A Dockerfile was created:

```
FROM nginx:latest
```

```
COPY index.html /usr/share/nginx/html/index.html
```

The Dockerfile used:

- FROM to select the Nginx base image.
- COPY to copy the custom webpage into the Nginx web directory.

Build Custom Image

```
sudo docker build -t mynginx:1.0 .
```

The custom Docker image was successfully created with the name:

```
mynginx:1.0
```

Verify Image

```
sudo docker images
```

The mynginx image was displayed in the list of Docker images.

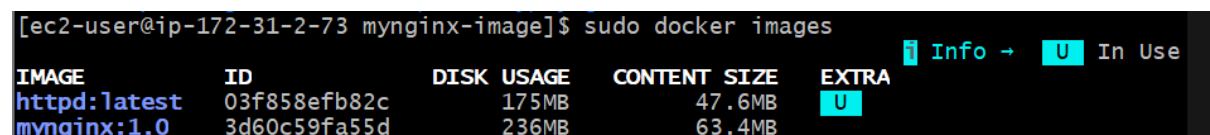


IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
httpd:latest	03f858efb82c	175MB	47.6MB	Info U In Use
mynginx:1.0	3d60c59fa55d	236MB	63.4MB	Info U In Use

6. Hosting the Custom Nginx Web Server

A container was created from the custom mynginx:1.0 image.

The container was initially created using port 8080. It was then stopped and removed so that a different host port could be used.

```
sudo docker stop mynginx  
sudo docker rm mynginx
```

The container was recreated using port 8081:

```
sudo docker run -d --name mynginx -p 8081:80 mynginx:1.0
```

Verify Container

```
sudo docker ps
```

The container was running with the following port mapping:

8081 → 80

This means:

- Port 8081 was used on the EC2 host.
- Port 80 was used by Nginx inside the container.

The AWS Security Group was configured to allow TCP port 8081.

The custom webpage was accessed using:

<http://13.201.129.84:8081>

Custom Nginx Webpage Hosted

The exact webpage displayed was:

Welcome to My Nginx Server

My Own Docker Image

This webpage is running from a custom Nginx Docker image.

Created as part of my Docker containerization lab.



7. Docker Volume

A Docker volume was created to understand persistent storage.

Create Docker Volume

```
sudo docker volume create mynginx-data
```

The volume was verified:

```
sudo docker volume ls
```

The volume mynginx-data was displayed.

Create Container Using the Volume

```
sudo docker run -d \  
  --name volume-test \  
  -p 8082:80 \  
  -v mynginx-data:/data \  
  nginx
```

Here:

- mynginx-data was the Docker volume.
- /data was the directory inside the container.
- The volume was mounted to /data.

8. Testing Data Persistence

A file was created inside the mounted volume:

```
sudo docker exec volume-test sh -c 'echo "This file is stored in a  
Docker volume." > /data/test.txt'
```

The file was verified:

```
sudo docker exec volume-test cat /data/test.txt
```

Output:

This file is stored in a Docker volume.

```
[ec2-user@ip-172-31-2-73 ~]$ sudo docker exec volume-test sh -c 'echo "This file is stored in a Docker volume." > /data/test.txt'
[ec2-user@ip-172-31-2-73 ~]$ sudo docker exec volume-test cat /data/test.txt
This file is stored in a Docker volume.
```

9. Removing the Original Container

The original container was stopped:

```
sudo docker stop volume-test
```

The container was removed:

```
sudo docker rm volume-test
```

The container was deleted, but the Docker volume mynginx-data was kept.

This demonstrated that the volume exists independently from the container.

10. Creating a New Container Using the Same Volume

A new Nginx container was created using the same Docker volume:

```
sudo docker run -d \
  --name volume-test-new \
  -p 8082:80 \
  -v mynginx-data:/data \
  nginx
```

The file from the previous container was checked:

```
sudo docker exec volume-test-new cat /data/test.txt
```

Output:

This file is stored in a Docker volume.

The file was still available even though the original container had been removed.

```
[ec2-user@ip-172-31-2-73 ~]$ sudo docker stop volume-test
volume-test
[ec2-user@ip-172-31-2-73 ~]$ sudo docker rm volume-test
volume-test
[ec2-user@ip-172-31-2-73 ~]$ sudo docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
3f7b2cf3186e	mynginx:1.0	"/docker-entrypoint..."	23 minutes ago	Up 23 minutes
00319f0464ac	httpd	"httpd-foreground"	41 minutes ago	Exited (0) 33 minutes ago

```
[ec2-user@ip-172-31-2-73 ~]$ sudo docker run -d \
  --name volume-test-new \
  -p 8082:80 \
  -v mynginx-data:/data \
  nginx
af4ea8f297a4b15a2c3e16c2a0955bb526f3cf55fbfb9543a1c861dda73a6410
[ec2-user@ip-172-31-2-73 ~]$ sudo docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
af4ea8f297a4	nginx	"/docker-entrypoint..."	8 seconds ago	Up 8 seconds
0.0.0.0:8082->80/tcp, [::]:8082->80/tcp	volume-test-new			
3f7b2cf3186e	mynginx:1.0	"/docker-entrypoint..."	24 minutes ago	Up 24 minutes
0.0.0.0:8081->80/tcp, [::]:8081->80/tcp	mynginx			

```
[ec2-user@ip-172-31-2-73 ~]$ sudo docker exec volume-test-new cat /data/test.txt
This file is stored in a Docker volume.
```

11. Important Docker Concepts Learned

Image

An image is a template used to create containers.

Example:

nginx:latest

Container

A container is a running instance created from an image.

Example:

mynginx

Dockerfile

A Dockerfile contains instructions for building a custom Docker image.

Example:

FROM nginx:latest

COPY index.html /usr/share/nginx/html/index.html

Port Mapping

Port mapping connects a host port to a container port.

Example:

8081:80

This means:

EC2 Host Port 8081



Container Port 80



Nginx Web Server

Volume

A volume stores data separately from the container.

In the practical:

mynginx-data

was mounted to:

/data

The data remained available after the original container was removed.

Conclusion

The Docker containerization practical was completed successfully. Docker was installed and configured on the RHEL 10.2 system, and both ready-made and custom Docker images were used to create containers.

An Apache web server was hosted using the official httpd image on port 8080, while a custom Nginx image was built using a Dockerfile and hosted on port 8081. Docker commands for managing images and containers were practiced. A Docker volume was also created and tested to demonstrate that data can persist independently of a container.